UCS 503 Software Engineering Project Report

CampusCycle

Smart Campus Bicycle Sharing and Mobility Management System

Submitted by:

102XXXXXXX Guneet Singh Nagi
102XXXXXXX Prabhnoor Oberoi

Group: XXXX BE Third Year, CSE

Submitted to:

Dr. Faculty Member Name

Mid-Semester Evaluation

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

1 Introduction **5**
- 1.1 Background and Context . . . 5
  - 1.1.1 Problem Statement . . . 5
- 1.2 Project Objectives . . . 6
- 1.3 Scope of This Report . . . 7

2 Methodology and Design **9**
- 2.1 Development Approach . . . 9
- 2.2 System Architecture . . . 9
  - 2.2.1 High-Level Design . . . 9
  - 2.2.2 Technical Stack . . . 10
- 2.3 Data Flow . . . 11
- 2.4 Database Design . . . 14
  - 2.4.1 Tables . . . 14
  - 2.4.2 Invariants Enforced in the Database . . . 14
  - 2.4.3 Migrations and Seed Data . . . 15
- 2.5 Implementation Details . . . 15
  - 2.5.1 Authentication and Access Control . . . 15
  - 2.5.2 Rentals and the Return Workflow . . . 16
  - 2.5.3 Campus Map and Station Availability . . . 16
  - 2.5.4 Ride History . . . 16
  - 2.5.5 Maintenance and the Safety Hold . . . 18
  - 2.5.6 Usage Analytics . . . 19
  - 2.5.7 Software-Based Geofencing . . . 21
- 2.6 Security Considerations . . . 25

3 Results and Evaluation **27**
- 3.1 Testing Methodology . . . 27
- 3.2 Results . . . 27
- 3.3 Scale of the Implementation . . . 28
- 3.4 Analysis . . . 29
  - 3.4.1 Were the Objectives Met? . . . 29
  - 3.4.2 Challenges and Their Resolution . . . 29

4 Conclusion **31**
- 4.1 Summary of Work . . . 31
- 4.2 Key Findings . . . 31
- 4.3 Limitations . . . 32

4.4 Future Work and Recommendations 32
4.5 Final Remarks 33

Bibliography **35**

Chapter 1

Introduction

1.1 Background and Context

A university campus is large enough that walking between a hostel, an academic block, the library and the sports complex consumes a noticeable part of a student's day, yet far too small for motorised transport to be worthwhile. This is the classic last-mile problem, and shared bicycles are a well-established answer to it: public bike-sharing schemes have been deployed in several hundred cities worldwide, and the research literature consistently identifies availability information and fleet maintenance as the factors that decide whether such a scheme is used or abandoned [9, 4].

A bicycle-sharing scheme, however, is not solved by simply buying bicycles. Without a system behind them, the fleet accumulates the familiar failures: cycles pile up at one station while another stands empty, a bicycle with failed brakes is rented out because nobody recorded the fault, cycles leave the campus and are not recovered, and the administration has no evidence on which to base decisions about where cycles should be placed.

CampusCycle addresses that gap. It is a web application through which a student locates an available bicycle, identifies it by scanning the QR code printed on it, rides between docking stations and returns it, while an administrator manages the fleet, tracks every ride, processes reported faults and reads usage analytics drawn from the live database.

1.1.1 Problem Statement

Design and build a centralised system for managing a campus bicycle-sharing service that:

1. tells a student, accurately and in real time, where a usable bicycle can be found;
2. records every rental from start to finish so that a cycle is never in two places at once and a student never holds two cycles at once;
3. ensures that a bicycle reported as unsafe cannot be rented out again until it has been inspected; and
4. gives the administration figures about fleet utilisation, station demand and maintenance load that are computed from the system's own records rather than estimated.

The difficulty in such a system is not any single feature. It is that several of these requirements are *invariants* rather than screens: statements that must hold no matter which part of the application is running, and no matter how many users act at the same time. Much of the design work reported here is concerned with where such invariants are enforced.

1.2 Project Objectives

The objectives below were set in the project proposal. Each is stated with the criterion by which it is judged, and the status at this evaluation.

1. **Account management and role separation.** Students register and sign in; an administrator console is reachable only by an administrator account. *Criterion:* every administrative endpoint rejects a student token. *Status: complete.*

2. **QR-based bicycle identification.** Each bicycle carries a unique QR code which the backend resolves to a bicycle record and its current state. *Criterion:* an unknown code and an unavailable cycle are both refused, with distinct responses. *Status: complete.*

3. **Rental and return workflow.** A student starts a ride, sees it in progress and returns the cycle at an approved station. *Criterion:* two students attempting the same cycle at the same instant produce exactly one rental. *Status: complete.*

4. **Interactive campus map.** Stations are shown on a map of the campus with their live availability. *Criterion:* the map and the station list never disagree, because both derive availability from one rule. *Status: complete.*

5. **Complete rental records.** Every ride is retained and can be inspected by the student who took it and by an administrator. *Criterion:* a student cannot read another student's ride, by any route. *Status: complete.*

6. **Fault reporting and maintenance management.** Students report faults; the administration works them through a defined lifecycle. *Criterion:* a cycle with an unresolved safety fault cannot be rented or returned to service. *Status: complete.*

7. **Usage analytics.** The administration sees fleet, station and ridership figures over a selectable period. *Criterion:* every figure is a database aggregate, and the page survives an empty database. *Status: complete.*

8. **Campus geofencing.** A configurable campus boundary, with violations recorded and surfaced for action. *Criterion:* a repeated breach does not

produce repeated alerts. *Status: complete as a software demonstration; see Section 2.5.7 for the limitation that matters.*

9. **Demand prediction and redistribution.** Prediction of station-level demand from historical rentals, and redistribution advice derived from it. *Status: deferred to the next phase*, for the reason given in Section 1.3.

1.3 Scope of This Report

Development was organised into phases. Phase 1 built the core rental system: accounts and roles, stations, the fleet, QR identification, the rental and return workflow, and an administrative dashboard. Phase 2, which this report covers in full, added the campus map, station availability, ride history, maintenance management, usage analytics and software geofencing.

The machine-learning demand-prediction module was deliberately held back rather than attempted alongside Phase 2. A demand model is only as good as the ridership history it is trained on, and that history is produced by the very modules Phase 2 delivers. Building the analytics layer first means the model will be trained on recorded data with a known schema rather than on synthetic figures. Both this report and the system therefore describe a complete two-phase implementation, with prediction scheduled as the next phase.

Chapter 2

Methodology and Design

2.1 Development Approach

The system was developed incrementally, in two phases with a working, tested application at the end of each. Phase 2 was implemented against the running Phase 1 system under three self-imposed constraints: the Phase 1 database was to be migrated rather than rebuilt, the existing architectural conventions were to be reused rather than replaced, and the Phase 1 test suite was to keep passing unchanged. The last of these functioned as a regression gate; the schema constraint is why all Phase 2 schema changes are additive and idempotent (Section 2.4.3).

2.2 System Architecture

2.2.1 High-Level Design

CampusCycle follows a three-tier architecture (Figure 2.1). The browser holds the entire user interface; the Node.js process holds all business rules; PostgreSQL holds all state. The two tiers communicate only over a JSON REST API, which means the server never produces markup and the browser never issues a query.

The application tier is deliberately layered, and a request always crosses the layers in the same order:

- **Routes** declare the URL, the HTTP verb and the roles permitted to reach the handler.
- **Middleware** verifies the JSON Web Token [5] and attaches the caller's identity and role to the request.
- **Controllers** read and validate request parameters and shape the response. They contain no business rules.
- **Services** hold the business rules, and own every database transaction.
- **Models** issue parameterised SQL and nothing else.

Keeping transactions in the service layer, rather than in controllers or models, is what makes the concurrency rules in Section 2.4.2 expressible at all: a rule that spans two tables has exactly one place it can live.

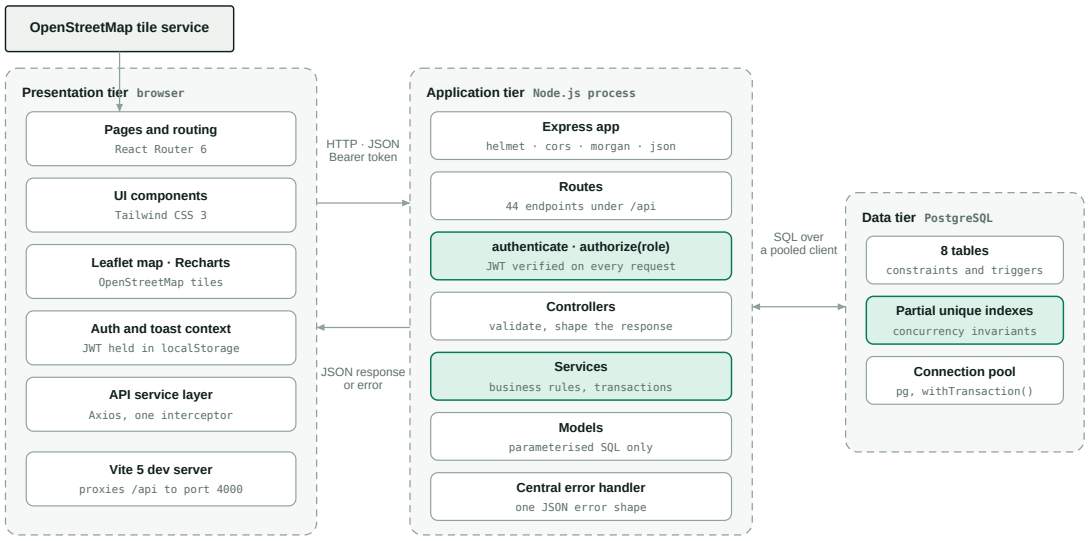


Figure 2.1: Three-tier architecture. Each request crosses the server layers in order, and only the model layer issues SQL.

2.2.2 Technical Stack

Departure from the proposal: PostgreSQL in place of MySQL

The proposal named MySQL. The implementation uses PostgreSQL, and the reason is specific rather than a matter of preference. Three of the system's invariants are enforced by *partial unique indexes* [7] — unique indexes that apply only to the rows matching a predicate:

- at most one `ACTIVE` rental per student;
- at most one `ACTIVE` rental per bicycle;
- at most one unresolved geofence alert per bicycle.

MySQL has no partial indexes, so each of these rules would have to be enforced in application code, where it is subject to the race conditions discussed in Section 2.4.2. PostgreSQL also supplies three features the analytics module relies on: `FILTER` clauses on aggregates, `generate_series` for producing a continuous date axis including days on which nothing happened, and `JSONB` for storing the geofence polygon. The cost of the change was confined to the schema files and the database driver; no application logic depends on the vendor.

Layer	Technology	Version
User interface	React with Vite	18.3 / 5.4
Styling	Tailwind CSS	3.4
Routing	React Router	6.24
HTTP client	Axios	1.7
Mapping	Leaflet + OpenStreetMap	1.9.4
Charts	Recharts	2.15
Server runtime	Node.js	≥ 18
Web framework	Express	4.19
Database	PostgreSQL	≥ 13
Database driver	node-postgres (pg)	8.12
Authentication	jsonwebtoken, bcryptjs	9.0 / 2.4
HTTP hardening	helmet, cors	7.1 / 2.8
Version control	Git and GitHub	—

Table 2.1: Implementation stack.

2.3 Data Flow

The data-flow diagrams use Gane–Sarson notation [3]. Figure 2.2 fixes the system boundary, Figure 2.3 decomposes the system into processes and stores, and Figures 2.5 and 2.9 explode the two processes whose internal behaviour is not obvious from the level-1 view.

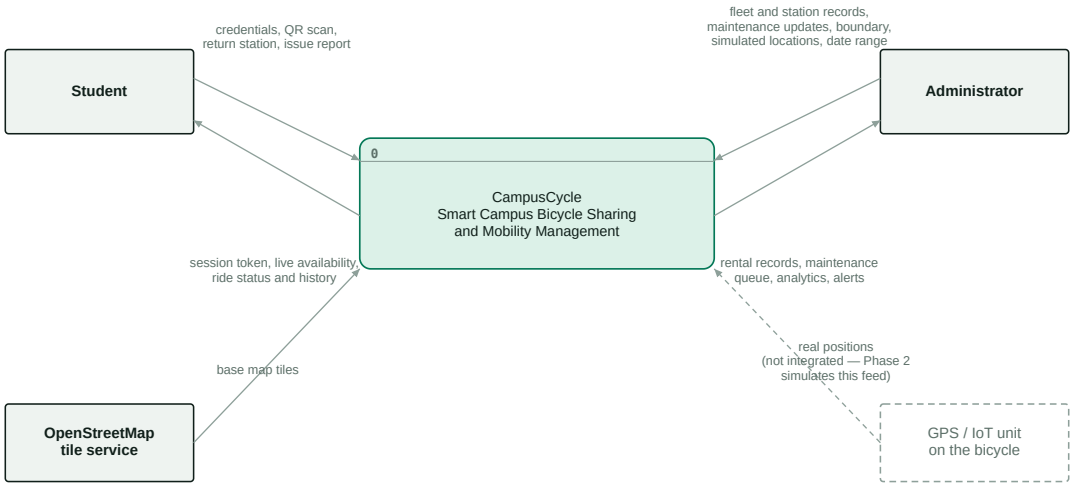


Figure 2.2: Context diagram. The dashed feed records an honest gap: the bicycles carry no GPS hardware, so every position the system holds was submitted by an administrator and stored as simulated.

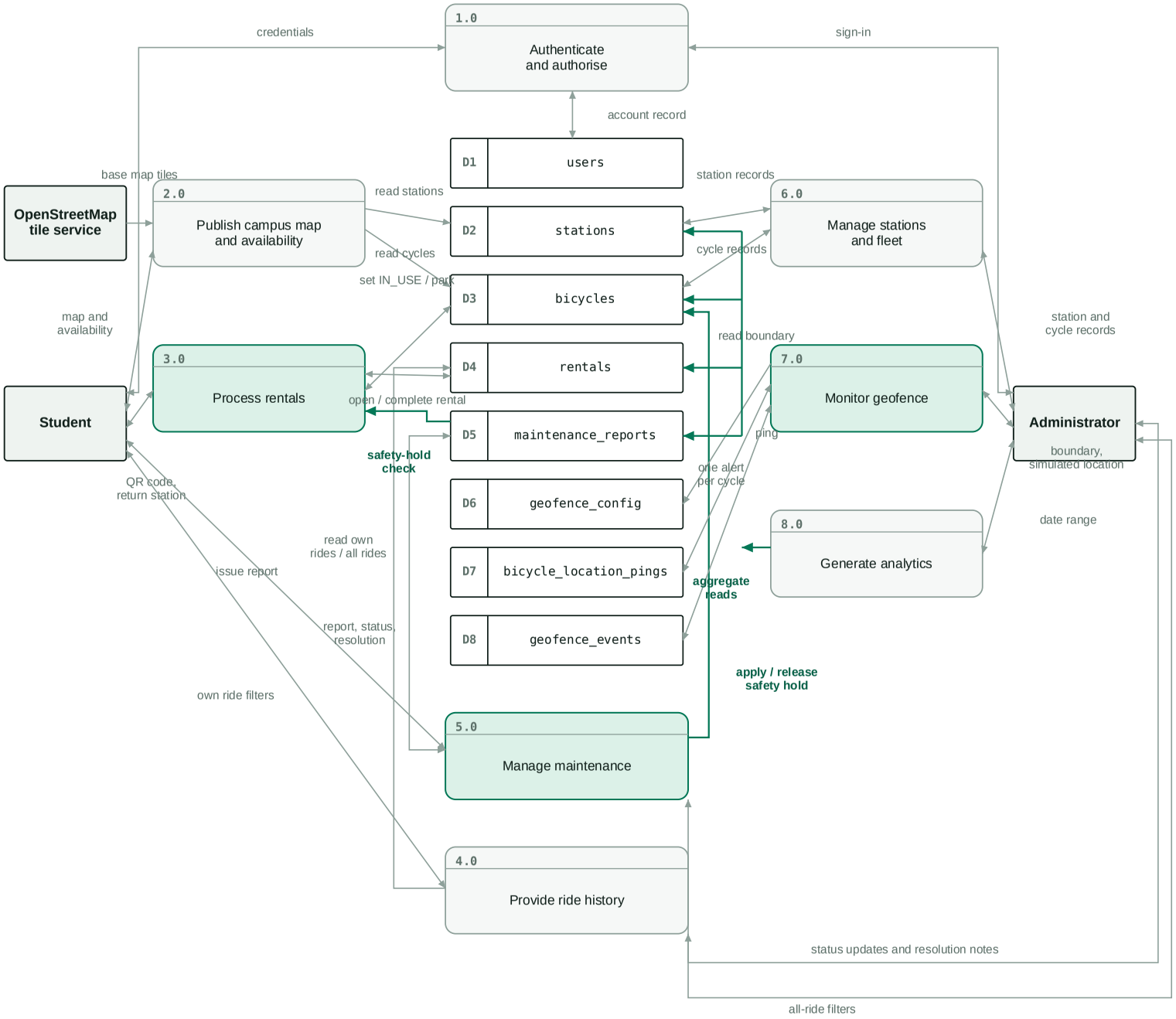


Figure 2.3: Level-1 data flow diagram. Eight processes around the eight tables they use. Writes to a store belong to exactly one process; reads cross freely.

2.4 Database Design

2.4.1 Tables

The schema is eight tables: four from Phase 1 and four added by Phase 2 (Table 2.2). Primary keys are `INTEGER GENERATED ALWAYS AS IDENTITY`, every enumerated column is constrained by a `CHECK` that mirrors a constant in the application code, and an `updated_at` trigger maintains modification timestamps.

Table	Phase	Contents
`users`	1	Accounts, bcrypt password hash [8], role
`stations`	1	Name, location, capacity, status; latitude and longitude added in Phase 2
`bicycles`	1	Cycle code, QR code, current station, status
`rentals`	1	Rider, cycle, start and end station, timestamps, status
`maintenance_reports`	2	Reported fault, severity, lifecycle status, reporter, resolution
`geofence_config`	2	Campus boundary polygon stored as JSONB
`bicycle_location_pings`	2	Every submitted position, with its source
`geofence_events`	2	Boundary violations, deduplicated per cycle

Table 2.2: Database tables.

2.4.2 Invariants Enforced in the Database

Rules that must hold under concurrent use are pushed into the database, because application-level checks of the form *read, decide, then write* are unsafe when two requests interleave between the read and the write [2]. The schema therefore carries:

- `uq_active_rental_per_user` and `uq_active_rental_per_bicycle` — partial unique indexes making a second active rental impossible rather than merely unlikely;

- `uq_geofence_open_event_per_bicycle` — the same device applied to alerts, which is what prevents a cycle left outside the boundary from generating an unbounded number of them;
- `uq_geofence_single_active` — at most one active boundary configuration;
- `chk_ended_at` and `chk_maintenance_resolved` — a completed ride must carry an end timestamp, and a resolved report a resolution time;
- `chk_stations_coords_paired` and range checks — a station has both coordinates or neither, and each lies within its valid range.

Above these, the service layer takes explicit row locks. Starting a rental, returning one, filing a report and changing a bicycle's status all run inside a transaction that holds `SELECT ... FOR UPDATE` on the bicycle row, so those four operations are serialised against each other for a given cycle. The database constraints remain as a second line of defence, and the unique-violation error codes they raise are translated into readable messages rather than surfacing as failures.

2.4.3 Migrations and Seed Data

Phase 2 could not rebuild the database, because a Phase 1 instance already held accounts and rental history. The schema change is therefore delivered as an additive, idempotent migration: it adds columns and tables, creates constraints only when they are absent, and never drops or truncates anything. Running it twice has the same effect as running it once. A separate, equally idempotent seed script fills in station coordinates and installs a campus boundary, and adds demonstration maintenance reports only when that table is still empty, so that a database holding genuine reports is left untouched.

2.5 Implementation Details

2.5.1 Authentication and Access Control

Passwords are hashed with bcrypt [8] at a configurable cost factor; the hash never leaves the database layer. A successful sign-in returns a JWT carrying the user id and role, which the browser attaches to every subsequent request.

Two decisions here are deliberate. First, public registration always creates a student: there is no input through which a caller can request the administrator role, which removes privilege escalation by construction. Second, and more pervasively, **the identity used for authorisation is always taken from the verified token, never from the request body or query string**. A request for ride history carries

no user id at all; the server derives it. This single convention is what makes objective 5 hold across every endpoint rather than endpoint by endpoint.

2.5.2 Rentals and the Return Workflow

A rental begins when a student submits a QR code. In one transaction, the service checks that the student holds no active ride, locks and loads the bicycle by its QR code, verifies that it is available and free of any safety hold, creates the rental, and marks the bicycle `IN_USE` with no station — a cycle in use is parked nowhere. The return reverses this: the ride is completed with an end station and timestamp, and the bicycle is parked at that station.

Capacity is enforced when an administrator assigns cycles to a station, but never on return. A student must always be able to end a ride; refusing a return because a station is notionally full would strand them.

2.5.3 Campus Map and Station Availability

Availability is defined once, in a single module, and every surface that displays it — the station list, the station detail page, the map, the student dashboard — derives its figures from that one definition:

- `available_count`: bicycles with status `AVAILABLE` parked at the station;
- `availability_pct`: `available_count` as a percentage of capacity, defined as zero when capacity is zero so that no screen can divide by zero;
- `availability_level`: one of `INACTIVE`, `NONE`, `LOW` or `GOOD`. A station is `LOW` when at most two cycles remain *or* fewer than 20% of its capacity is available — the count catches a nearly-empty large station, the percentage a nearly-empty small one.

The map itself is drawn with Leaflet [1] over OpenStreetMap tiles [6] (Figure 2.4). Markers are generated from the station data, so a marker always carries the current count; the frontend contains no availability figures of its own. Stations that have no coordinates are reported explicitly rather than silently omitted.

2.5.4 Ride History

Ride durations are computed by PostgreSQL from the stored `TIMESTAMPTZ` values rather than in the browser: a completed ride reports `ended_at − started_at`, an active one reports `NOW() − started_at`, and a ride with a missing or inconsistent timestamp reports no duration at all rather than a negative or invented one. Administrators additionally get filters by date range, cycle, station and status, with pagination; no endpoint returns an unbounded result set.

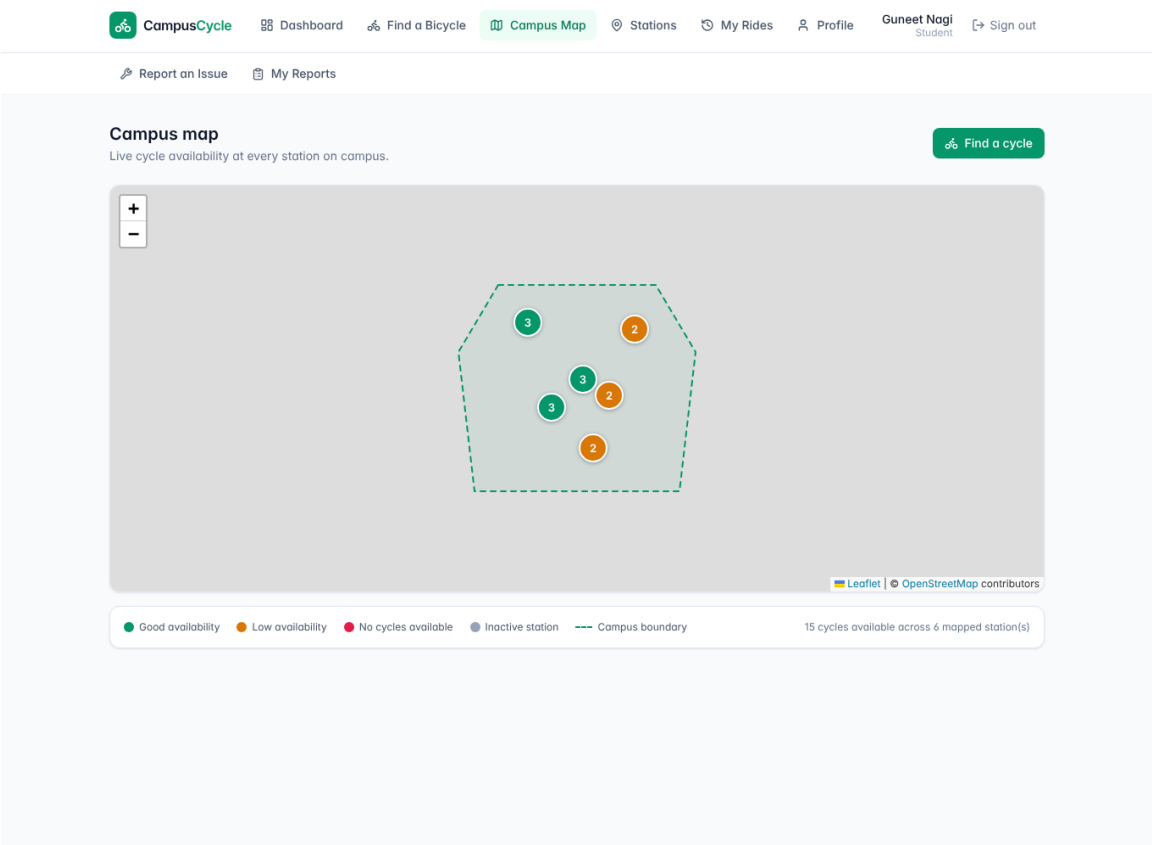


Figure 2.4: Student campus map: one marker per station, coloured and numbered by live availability, with the configured campus boundary drawn as a dashed outline. The basemap tiles are fetched from OpenStreetMap at run time and are absent in this capture, which was taken on a host without external network access.

2.5.5 Maintenance and the Safety Hold

This module is the one place where a user-facing feature and a safety property meet, and Figure 2.5 shows the resulting flow in full.

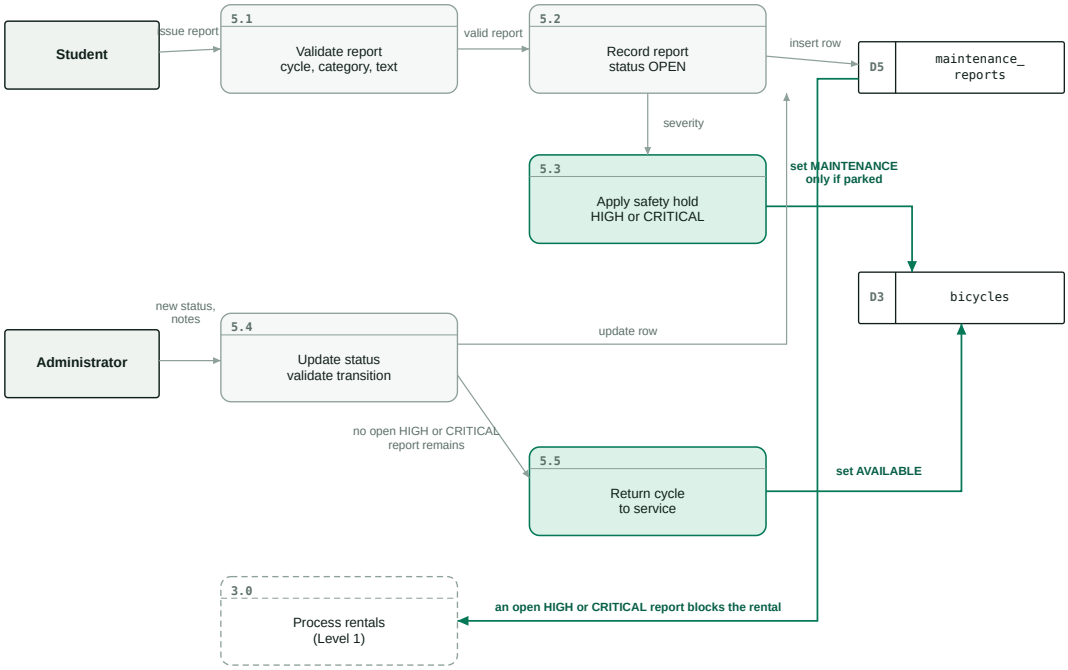


Figure 2.5: Maintenance and the safety hold. The highlighted path is the invariant: an unresolved high-severity report blocks the rental process and blocks any return to service.

A student reports a fault against a cycle with a category, a description and a severity (Figure 2.6). Severity drives everything that follows:

- A `HIGH` or `CRITICAL` report places the cycle on a **safety hold**. If the cycle is parked it is moved to `MAINTENANCE` immediately. If it is out on a ride the rider is not interrupted — taking a cycle out from under someone mid-ride would be worse than the fault — and the hold is applied when it is returned, so it is parked as `MAINTENANCE` rather than `AVAILABLE`.
- A `LOW` or `MEDIUM` report is recorded for review and leaves the cycle in service.

While a hold exists the cycle cannot be rented, cannot be parked as available on return, and cannot be forced back into service by an administrator. All three routes consult the same check. Reports move through `OPEN` → `IN_PROGRESS` → `RESOLVED` →

`CLOSED`, with reopening permitted from the two middle states and `CLOSED` terminal; invalid transitions are refused, and resolution notes are required to resolve or close a report (Figure 2.7).

Resolving a report does *not* by itself return the cycle to service. Clearing a report is a statement about a record; returning a cycle to service is a statement about a physical object that someone must have actually repaired and parked. The system therefore requires a separate, explicit administrative action, and permits it only once no hold remains.

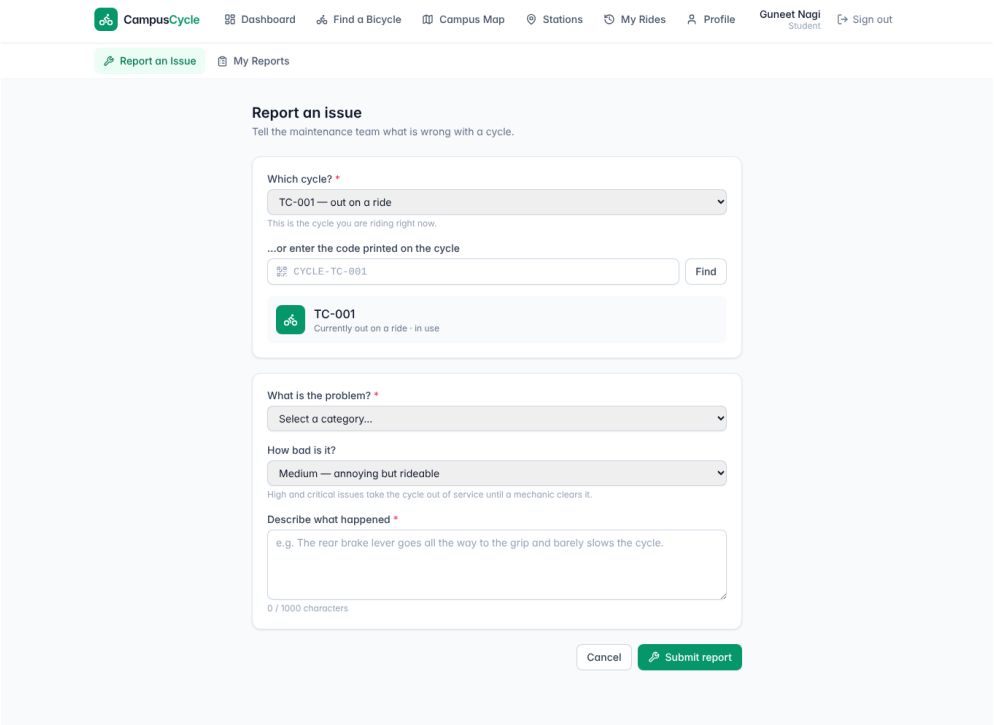


Figure 2.6: Fault reporting. The cycle is pre-selected from the student's active ride, and the form warns that a safety-severity report takes the cycle out of service.

2.5.6 Usage Analytics

Every figure on the analytics dashboard (Figure 2.8) is a PostgreSQL aggregate over the live tables; nothing is cached or hard-coded. The page separates two kinds of number that are easily confused: a *snapshot* of the fleet together with all-time totals, which the date filter does not affect, and *period* figures counting only rides that started inside the selected range.

Three details were necessary for the figures to be trustworthy. Empty days are generated with `generate_series` and joined against, so a quiet week produces zeros

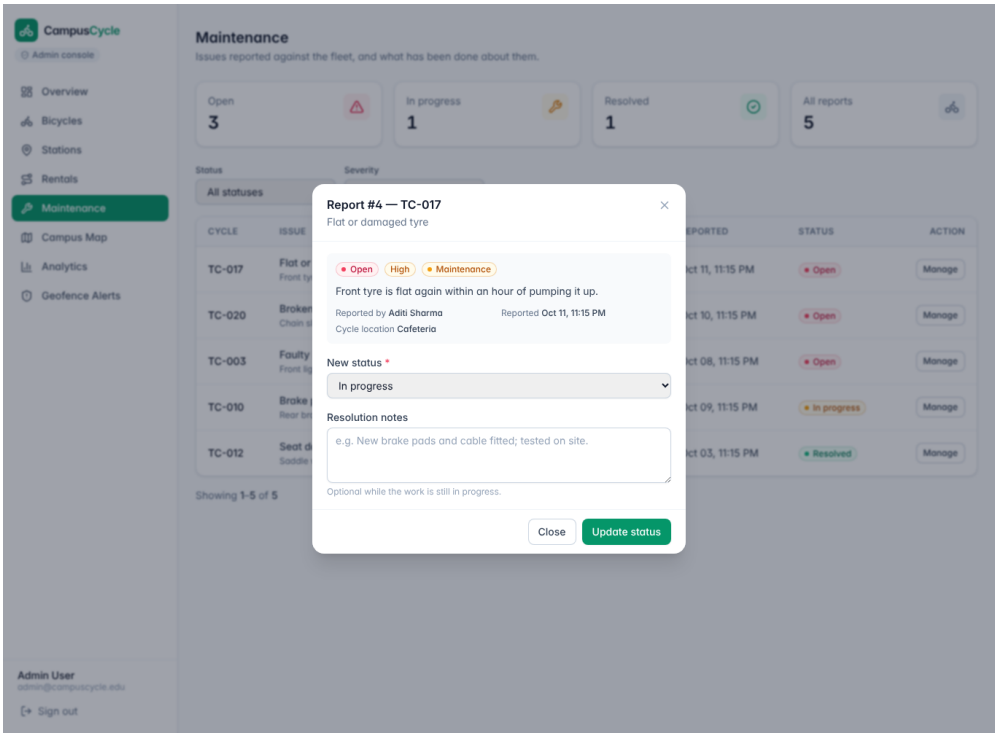


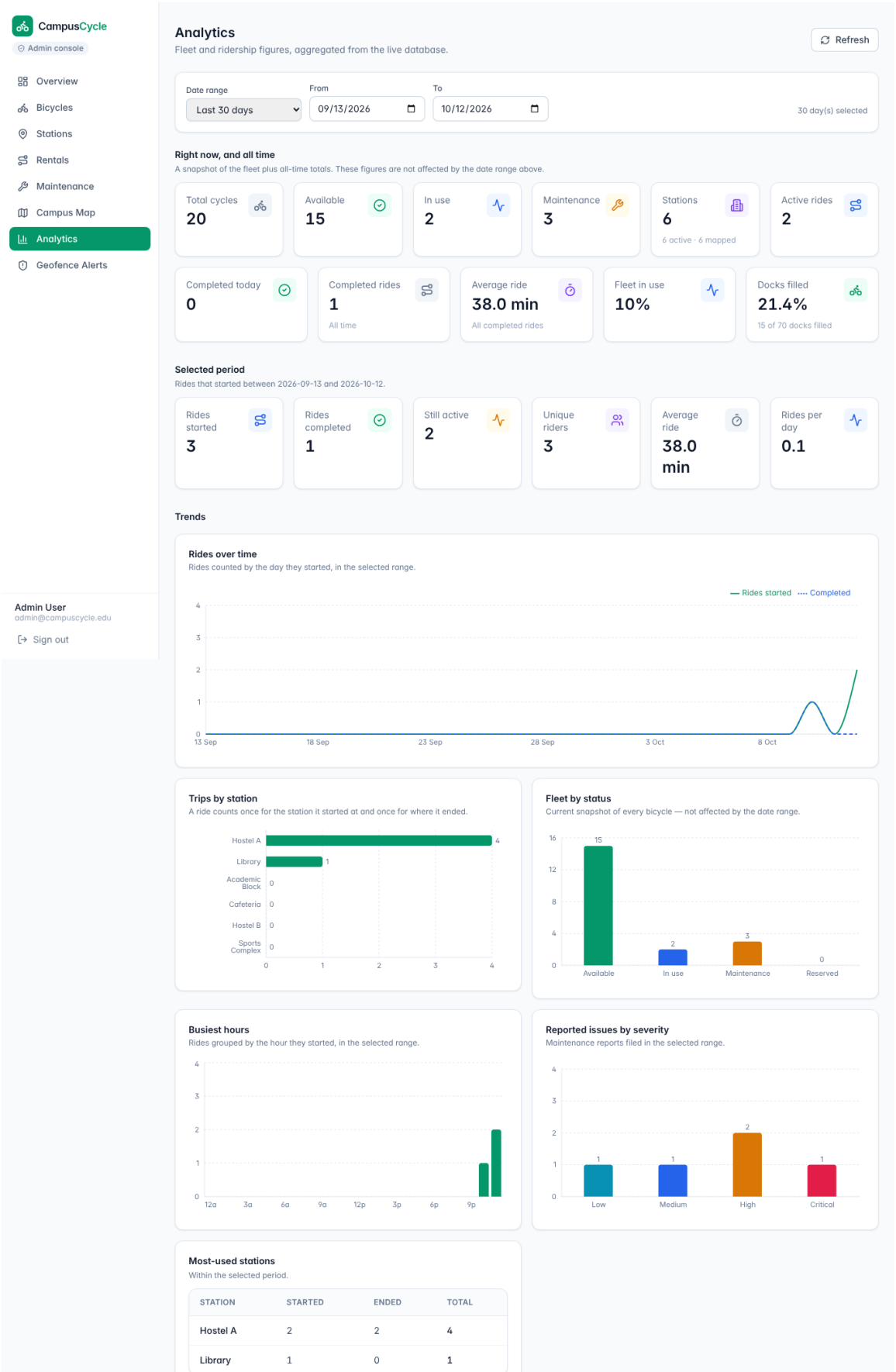
Figure 2.7: Administrative maintenance queue. Only the transitions permitted from the report's current state are offered.

on a continuous axis rather than a gap. Averages over an empty set return no value rather than zero, which are different statements. Day and hour buckets are computed in an explicitly configured reporting timezone, because `date_trunc` otherwise follows the database session's timezone: during development this placed rides on the wrong calendar day whenever the server session ran in UTC while the campus is at UTC+05:30.

2.5.7 Software-Based Geofencing

The bicycles carry no GPS hardware. There is no continuous tracking in this system, and the application does not present one. Every position it holds was explicitly submitted through an administrator-only endpoint and is stored with its source; only simulated positions are accepted, and a position claiming to come from a device is rejected until a real integration exists. A student's browser location is never accepted as a bicycle's position, which is why the endpoint takes an explicit cycle identifier and is closed to students.

The campus boundary is a configurable polygon stored as JSONB and tested with a ray-casting point-in-polygon algorithm [10]; at the scale of one campus, treating latitude and longitude as a plane introduces no meaningful error. A position outside the boundary opens an alert recording the cycle, the time, the coordinates and the ride in progress. A *repeated* breach does not open another: it increments the existing alert's counter and updates its last known position, a rule enforced by the partial unique index named earlier. A cycle seen back inside records that fact but does not close the alert, because whether the situation is resolved is a judgement for a person (Figure 2.10).


CampusCycle
Admin console
Overview
Bicycles
Stations
Rentals
Maintenance
Campus Map
Analytics
Geofence Alerts
Admin User
admin@campuscycle.edu
Sign out
Analytics
Fleet and ridership figures, aggregated from the live database.
Refresh
Date range
Last 30 days
From
09/13/2026
To
10/12/2026
30 day(s) selected
Right now, and all time
A snapshot of the fleet plus all-time totals. These figures are not affected by the date range above.
Total cycles
20
Available
15
In use
2
Maintenance
3
Stations
6
6 active · 6 mapped
Active rides
2
Completed today
0
Completed rides
1
All time
Average ride
38.0 min
All completed rides
Fleet in use
10%
Docks filled
21.4%
15 of 70 docks filled
Selected period
Rides that started between 2026-09-13 and 2026-10-12.
Rides started
3
Rides completed
1
Still active
2
Unique riders
3
Average ride
38.0 min
Rides per day
0.1
Trends
Rides over time
Rides counted by the day they started, in the selected range.
Rides started
Completed
13 Sep
18 Sep
23 Sep
28 Sep
3 Oct
8 Oct
Trips by station
A ride counts once for the station it started at and once for where it ended.
Hostel A
Library
Academic Block
Cafeteria
Hostel B
Sports Complex
Fleet by status
Current snapshot of every bicycle — not affected by the date range.
Available
In use
Maintenance
Reserved
Busiest hours
Rides grouped by the hour they started, in the selected range.
12a
3a
6a
9a
12p
3p
6p
9p
Reported issues by severity
Maintenance reports filed in the selected range.
Low
Medium
High
Critical
Most-used stations
Within the selected period.
STATION
STARTED
ENDED
TOTAL
Hostel A
2
2
4
Library
1
0
1

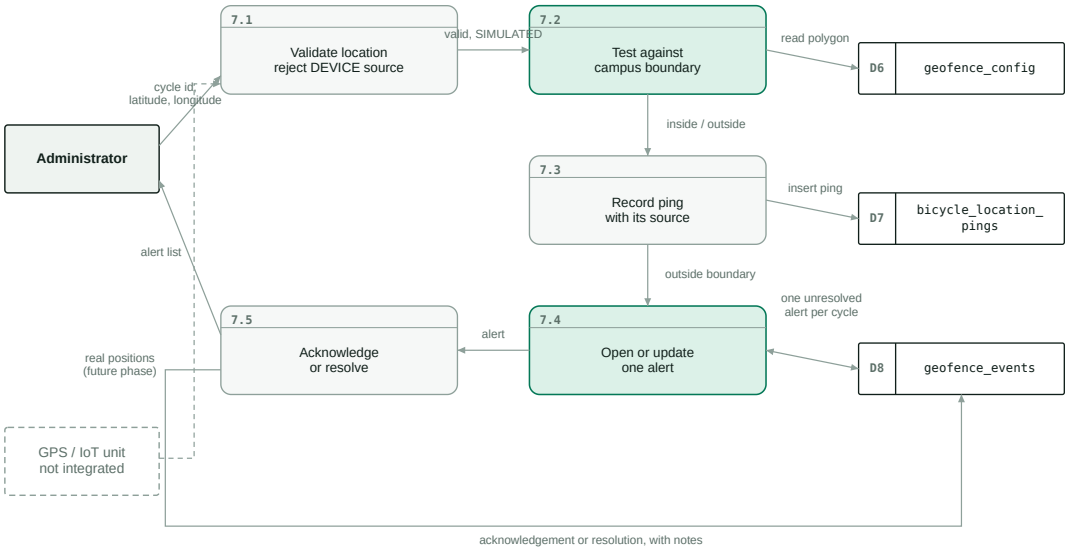


Figure 2.9: Geofencing. The source is validated before anything else, and one unresolved alert per cycle is what keeps a wandering bicycle from flooding the alert list.

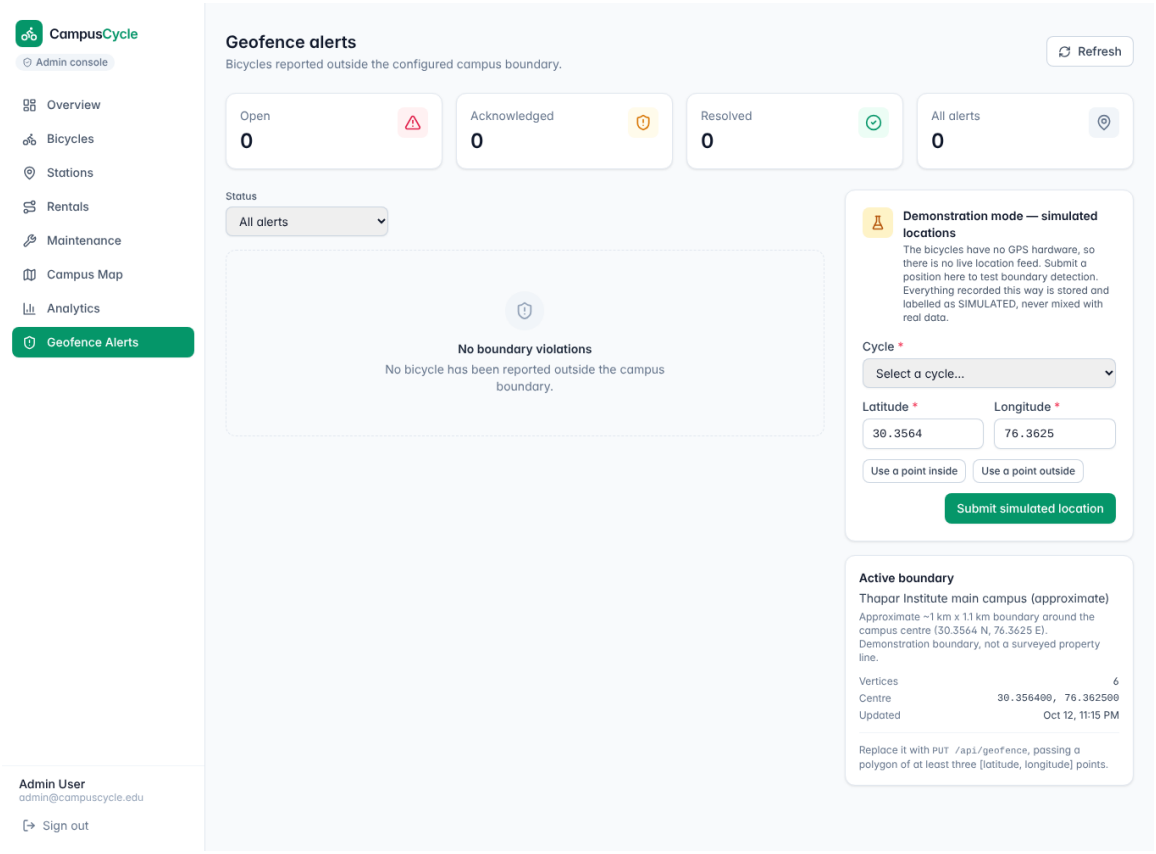


Figure 2.10: Geofence alerts, with the clearly labelled demonstration panel used to submit simulated positions.

2.6 Security Considerations

- **Credentials.** Passwords are stored only as bcrypt hashes. Sign-in failures return one uniform message, so the response cannot be used to discover which email addresses are registered.
- **Authorisation.** Role checks sit in middleware on the route, not in the handler, so an endpoint cannot be added without a decision about who may reach it.
- **Injection.** Every query is parameterised; no SQL is assembled from user input.
- **Secrets.** The JWT secret and database credentials are read from the environment, and the environment file is excluded from version control. A test asserts that no password hash, token secret or connection string appears in any API response.
- **Transport and headers.** `helmet` sets standard security headers and CORS is restricted to the configured origin.

Chapter 3

Results and Evaluation

3.1 Testing Methodology

Testing is automated and runs against a real PostgreSQL database rather than a mocked one, because most of the properties worth testing in this system — uniqueness under concurrency, transactional consistency, the correctness of aggregate queries — exist only in the database and would be assumed away by a mock.

- **Integration and end-to-end testing.** Two suites rebuild the schema, apply the migrations, load the seed data, start the Express application on an ephemeral port and drive it over real HTTP. They cover the full rental lifecycle, role-based access control, the maintenance and safety-hold workflow, analytics figures checked against direct SQL counts, and geofence detection and de-duplication.
- **Concurrency testing.** Two sign-ins issue simultaneous requests for the same bicycle; the suite asserts that exactly one succeeds and one is refused.
- **Regression testing.** The Phase 1 suite is run unchanged after every Phase 2 change. Its one modification was to apply the new migrations before seeding; no assertion was altered or removed.
- **User-interface testing.** A browser test drives the real interface end to end: a student files a critical fault report, the cycle disappears from the list of rentable cycles, an administrator resolves the report and is offered the return-to-service action, then simulates a boundary breach and confirms that repeating it produces no second alert.
- **Build verification.** The production frontend build is run as part of the check, so a broken import or unused dependency fails immediately.

3.2 Results

A selection of the properties asserted, chosen because each corresponds to an objective rather than to a screen:

- Two students requesting the same bicycle simultaneously produce exactly one rental and one refusal.

Suite	Checks	Passed	Covers
Phase 1 end-to-end	25	25	Rental lifecycle, RBAC, concurrency
Phase 2 end-to-end	139	139	Map, history, maintenance, analytics, geofencing, access control
Browser flow	17	17	Student and administrator journeys
Total	**181**	**181**	

Table 3.1: Automated test results. All suites pass with no failures.

- A seeded 38-minute ride reports a duration of exactly 2280 seconds; a ride still in progress reports a duration that advances.
- A student requesting another student's ride receives 403; an unknown ride identifier receives 404; a malformed one receives 400.
- A cycle with an unresolved critical report cannot be rented, and an administrator attempting to mark it available receives 409.
- Three successive out-of-bounds positions produce one alert with a count of three, not three alerts.
- Every administrative endpoint returns 403 for a student token and 401 for no token.
- No password hash, token secret or connection string appears in any API response.

3.3 Scale of the Implementation

Measure	Value
Backend source files	53
Backend lines of code	5,438
Frontend source files	64
Frontend lines of code	7,528
SQL (schema, migration, seeds)	551
REST endpoints	44
Database tables	8
Indexes and unique constraints	20
Automated checks	181

Table 3.2: Implementation size at this evaluation.

3.4 Analysis

3.4.1 Were the Objectives Met?

Eight of the nine objectives are complete and evidenced by the tests listed above. The ninth, demand prediction, was deliberately deferred for the reason given in Section 1.3.

One objective deserves a qualified answer. The geofencing objective is met as specified — a boundary is configurable, violations are detected, recorded, de-duplicated and actionable — but it operates on submitted positions rather than measured ones, because the project has no GPS hardware. Rather than let the interface imply a tracking capability that does not exist, the limitation is made explicit in the API, in the stored data and on screen. The software is structured so that the eventual hardware feed replaces one validated input path and nothing else.

3.4.2 Challenges and Their Resolution

Concurrent access to a single bicycle. The first implementation checked a bicycle's availability and then created the rental, which is correct when tested by one person and wrong under load: two requests can both pass the check before either writes. The resolution was to stop treating this as application logic. The bicycle row is locked for the duration of the transaction, and a partial unique index makes a second active rental impossible even if the lock were somehow bypassed. The concurrency test exists to prove the fix rather than to illustrate the feature.

Keeping a safety rule true everywhere. A bicycle can become available through three different paths: an administrator sets its status, a ride ends, or it is created. An early version checked for unresolved faults only in the first. The resolution was to make the safety hold a single function consulted by all three paths, and to express

it as a property of the maintenance records rather than as an extra column on the bicycle — a flag would have been one more piece of state capable of disagreeing with the reports it summarises.

Timezone-dependent analytics. Rides were being attributed to the wrong calendar day. The cause was that day boundaries were computed in Node using the machine's local timezone while PostgreSQL grouped them using the database session's timezone. The resolution was to make the reporting timezone an explicit configuration value used on both sides, so that "today" means one thing throughout the system.

Migrating a database already in use. The schema had to change without discarding Phase 1 data. The resolution was to separate the destructive development path from the additive production path: the base schema remains re-runnable for a fresh install, while the Phase 2 migration adds only what is missing and can be run repeatedly without effect.

Chapter 4

Conclusion

4.1 Summary of Work

CampusCycle is a working campus bicycle-sharing system. Students sign in, find an available cycle on a live map or in a filtered list, identify it by its QR code, ride it between stations, return it, review their ride history and report faults. Administrators manage stations and the fleet, inspect every ride, work reported faults through a maintenance lifecycle, read usage analytics computed from the live database and act on geofence alerts.

Across two phases the system comprises 44 REST endpoints over eight PostgreSQL tables, with 181 automated checks passing against a real database. Phase 2 was delivered without modifying a single Phase 1 assertion and without rebuilding the Phase 1 database.

4.2 Key Findings

- **Invariants belong in the database.** Every rule that had to survive concurrent use — one active ride per student, one per cycle, one unresolved alert per bicycle — is enforced by a partial unique index. Application code states the rule clearly and produces a readable error; the database guarantees it.

- **A rule enforced in one place is a rule; enforced in three, it is a coincidence.** The safety hold became reliable only once every path that could make a bicycle available consulted the same check. The same applies to availability: one definition is why the map and the station list cannot disagree.

- **Authorisation must not be derived from the request.** Taking the user's identity from the verified token rather than from a request parameter makes "a student sees only their own rides" a property of the system rather than a property of each endpoint.

- **An honest limitation is cheaper than a convincing one.** Labelling simulated positions as simulated, at every layer, cost a column and a validation rule. It leaves the system extensible to real hardware and prevents an operator from ever mistaking a demonstration for evidence.

4.3 Limitations

- There is no real-time tracking; bicycle positions are administrator-submitted and simulated.
- There is no demand prediction or redistribution advice; that is the next phase.
- Rides are free. No pricing, wallet or payment integration exists.
- The map requires internet access for OpenStreetMap tiles; without it, markers and the boundary still render over a blank background.
- Station coordinates place the demonstration stations within the campus but are approximate rather than surveyed, and the boundary polygon approximates the campus extent rather than the legal property line.
- There is no password-reset or email verification flow.

4.4 Future Work and Recommendations

1. **Demand prediction.** Train a regression or tree-based model on the recorded rental history using station, hour of day, day of week and recent demand as features. The analytics layer already produces these aggregates, so the model consumes a stable schema rather than ad-hoc extraction.
2. **Redistribution advice.** Compare predicted demand against current station occupancy and recommend movements. This requires no new data, only the model from the previous item.
3. **GPS and IoT integration.** Fit tracking units and allow device-sourced positions. The ingestion path, the storage, the boundary test and the alerting already exist; the change is to admit a second value in the source column and to accept authenticated device traffic.
4. **Electronic locks.** With trusted hardware the QR workflow could unlock a physical lock, closing the gap between a rental record and physical possession of the cycle.
5. **Payments and membership.** Semester passes, penalties for late returns and receipts, should the service move beyond free use.
6. **Deployment hardening.** Refresh tokens with rotation, rate limiting on authentication, structured logging, automated backups and a continuous integration pipeline running the existing suites on every push.

4.5 Final Remarks

The most useful lesson from this project was about where correctness lives. The features that were straightforward to build were the ones that are visible: a form, a list, a chart. The ones that took real design effort were invisible when working and only noticeable when broken — two students must not get the same bicycle, a cycle reported unsafe must not reappear as available, a student must not see another student's rides. Each of these was first written as application logic, and each became reliable only once it was pushed down to a single enforcement point, usually a database constraint, with the application code reduced to stating the rule clearly and explaining the refusal.

The second lesson concerns honesty in engineering. It would have been easy to present the geofencing module as bicycle tracking, since the screens would look identical. Recording the provenance of every position and refusing to accept a simulated one as real cost very little and leaves both the system and its documentation truthful about what has actually been built — which is also what makes the hardware phase a straightforward extension rather than a correction.

Bibliography

[1] Vladimir Agafonkin. *Leaflet: An Open-Source JavaScript Library for Mobile-Friendly Interactive Maps*, 2024.

[2] Philip A. Bernstein, Vassos Hadzilacos, and Nathan Goodman. *Concurrency Control and Recovery in Database Systems*. Addison-Wesley, 1987.

[3] Tom DeMarco. Structured analysis and system specification. *Yourdon Press Computing Series*, 1979.

[4] Elliot Fishman. Bikeshare: A review of recent literature. *Transport Reviews*, 36(1):92–113, 2016.

[5] Michael B. Jones, John Bradley, and Nat Sakimura. JSON Web Token (JWT). RFC 7519, Internet Engineering Task Force, 2015.

[6] OpenStreetMap Contributors. Openstreetmap, 2024.

[7] PostgreSQL Global Development Group. *PostgreSQL Documentation: Partial Indexes*, 2024.

[8] Niels Provos and David Mazières. A future-adaptable password scheme. In *Proceedings of the USENIX Annual Technical Conference*, pages 81–91, 1999.

[9] Susan A. Shaheen, Stacey Guzman, and Hua Zhang. Bikesharing in europe, the americas, and asia: Past, present, and future. *Transportation Research Record*, 2143(1):159–167, 2010.

[10] Michael Ian Shamos and Dan Hoey. Geometric intersection problems. *17th Annual Symposium on Foundations of Computer Science*, pages 208–215, 1976.